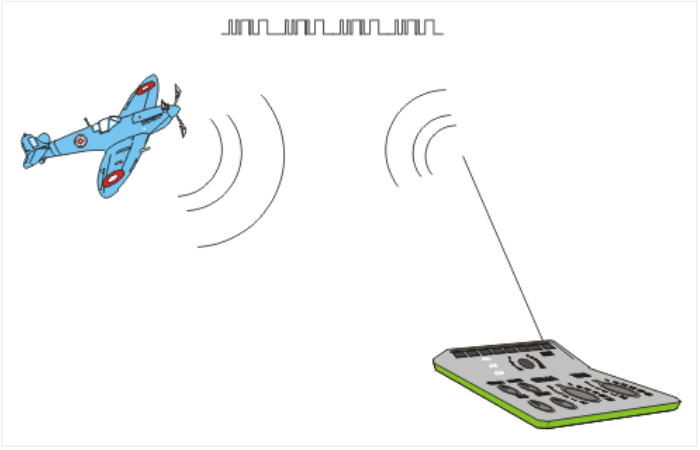
3.8 SERIAL COMMUNICATION MODULES

The USART is one of the oldest serial communication systems. The modern versions of this system are upgraded and called somewhat differently – EUSART.

EUSART

The *Enhanced Universal Synchronous Asynchronous Receiver Transmitter* (EUSART)*module* is a serial I/O communication peripheral unit. It is also known as *Serial Communications Interface* (SCI). It contains all clock generators, shift registers and data buffers necessary to perform an input/output serial data transfer independently of the device program execution. As its name states, apart from using the clock for synchronization, this module can also establish asynchronous connection, which makes it unique for some of the applications. For example, in the event that it is difficult or impossible to provide special channels for clock and data transfer (for example, radio or infrared remote control), the EUSART module is definitely the best possible solution.



The EUSART system integrated into the PIC16F887 microcontroller has the following features:

- *Full-duplex* asynchronous transmit and receive;
- Programmable 8- or 9-bit wide characters;
- Address detection in 9-bit mode;
- Input buffer overrun error detection; and
- *Half-duplex* communication in synchronous mode.

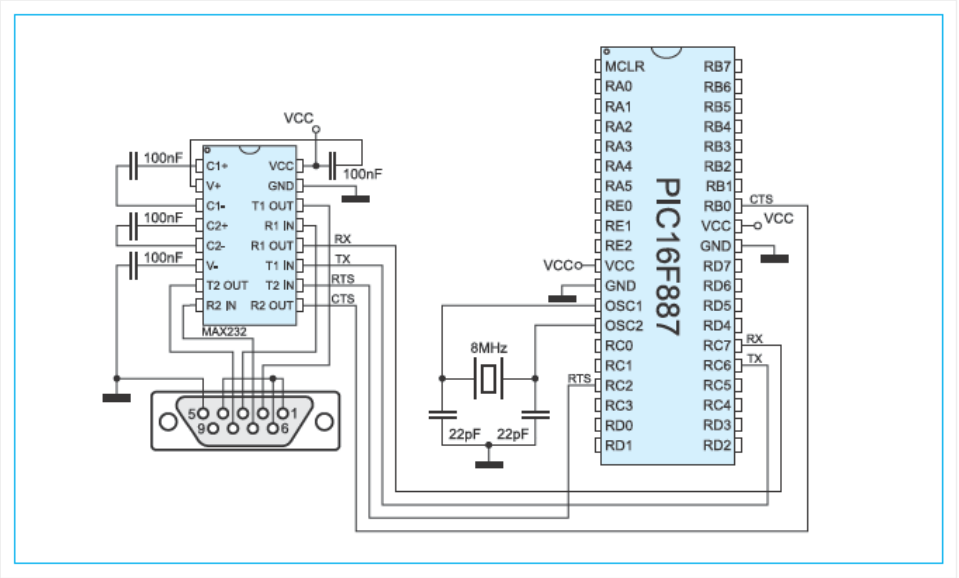
EUSART ASYNCHRONOUS MODE

The EUSART transmits and receives data using a standard non-return-to-zero (NRZ) format. As seen in figure below, this mode doesn’t use clock signal, while the format of data being transferred is very simple:

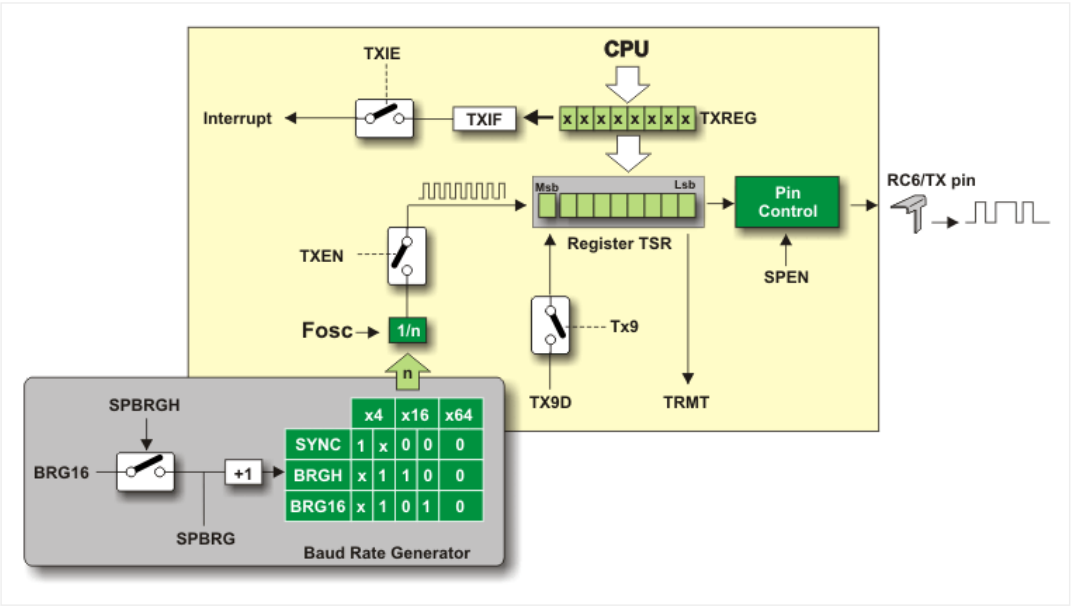
Briefly, each data is transferred in the following way:

- In idle state, data line has high logic level (1);
- Each data transmission starts with the START bit which is always a zero (0);
- Each data is 8- or 9-bit wide (the LSB bit is transferred first); and
- Each data transmission ends with the STOP bit which is always a one (1).

Figure below shows a common way of connecting PIC microcontroller that uses EUSART module. The RS-232 circuit is used as a voltage level converter.



EUSART ASYNCHRONOUS TRANSMITTER



In order to enable data transmission via EUSART module, it is necessary to configure it to operate as a transmitter. In other words, it is necessary to define the state of the following bits:

- **TXEN = 1** – EUSART transmitter is enabled by setting the TXEN bit of the TXSTA register.
- **SYNC = 0** – EUSART is configured to operate in asynchronous mode by clearing the SYNC bit of the TXSTA register.
- **SPEN = 1** – By setting the SPEN bit of the RCSTAregister, EUSART is enabled and the TX/CK pin is

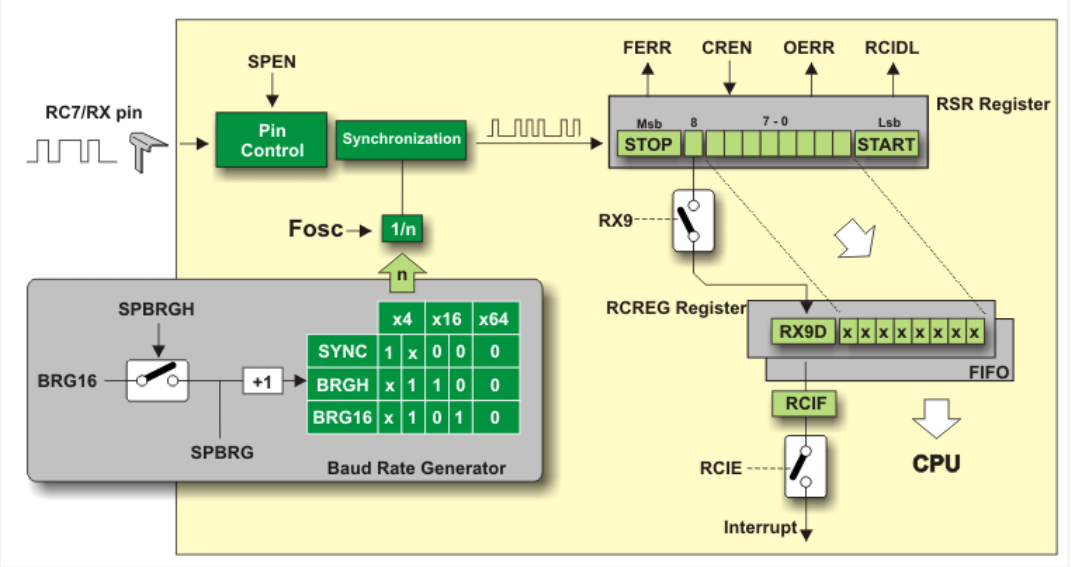
automatically configured as an output. If this bit is simultaneously used for some analogue function, it must be disabled by clearing the corresponding bit of the ANSEL register.

The central part of the EUSART transmitter is the shift register TSR which is not directly accessible by the user. In order to start data transfer, the module must be enabled by setting the TXEN bit of the TXSTA register. Data to be sent should be written to the TXREG register, which will cause the following sequence of events:

- Byte will be immediately transferred to the shift register TSR;
- TXREG register remains empty, which is indicated by setting the flag bit TXIF of the PIR1 register. If the TXIE bit of the PIE1 register is set, an interrupt will be generated. However, the flag is set regardless of whether an interrupt is enabled or not and it cannot be cleared by software, but by writing new data to the TXREG register.
- Control electronics 'pushes' data toward the TX pin in synchronization with internal clock: START bit (0) … data … STOP bit (1).
- When the last bit leaves the TSR register, the TRMT bit of the TXSTA register is automatically set.
- If the TXREG register has received a new character data in the meantime, the whole procedure will be immediately repeated after the STOP bit of the previous character has been transmitted.

9-bit data transfer is enabled by setting the TX9 bit of the TXSTA register. The TX9D bit of the TXSTA register is the ninth and most significant data bit. When transferring 9-bit data, the TX9D data bit must be written prior to writing the 8 least significant bits into the TXREG register. All nine bits of data will be transferred to the TSR shift register immediately after the TXREG write is complete.

EUSART ASYNCHRONOUS RECEIVER



In order to enable data transmission via EUSART module, it is necessary to configure it to operate as a transmitter. In other words, it is necessary to define the state of the following bits:

- **CREN = 1** – EUSART receiver is enabled by setting the CREN bit of the RCSTA register;
- **SYNC = 0** – EUSART is configured to operate in asynchronous mode by clearing the SYNC bit stored in the TXSTA register; and
- **SPEN = 1** – By setting the SPEN bit of the RCSTAregister, EUSART is enabled and the RX/DT pin is automatically configured as an input. If this bit is simultaneously used for some analogue function, it must be disabled by clearing the corresponding bit of the ANSEL register.

When this first and necessary step is accomplished and the START bit is detected, data is transferred to the shift register RSR through the RX pin. When the STOP bit has been received, the following occurs:

- Data is automatically transferred to the RCREG register (if empty);
- The flag bit RCIF is set and an interrupt, if enabled by the RCIE bit of the PIE1 register, occurs. Similarly to the

transmitter, the flag bit is cleared by software only, i.e. by reading the RCREG register. Bear in mind that this is a two character FIFO memory (*first-in*, *first-out*) which allows reception of two characters simultaneously;
- If the RCREG register is occupied (contains two bytes) and the shift register detects new STOP bit, the overflow bit OERR will be set. In this case, a new coming data is lost, and the OEER bit must be cleared by software. It is done by clearing and resetting the CREN bit;
  Note: it is not possible to receive new data as far as the OERR bit is set.
- If the STOP bit is a zero (0), the FERR bit of the RCSTA register detecting receive error will be set; and
- To enable 9-bit data reception, it is necessary to set the RX9 bit of the RCSTA register.

RECEIVE ERROR DETECTION

There are two types of errors which the microcontroller can automatically detect. The first one is called *Framing error* and occurs when the receiver does not detect the STOP bit at the expected time. Such an error is indicated by the FERR bit of the RCSTA register. If this bit is set, the last received data may be incorrect. Here are several things important to know:

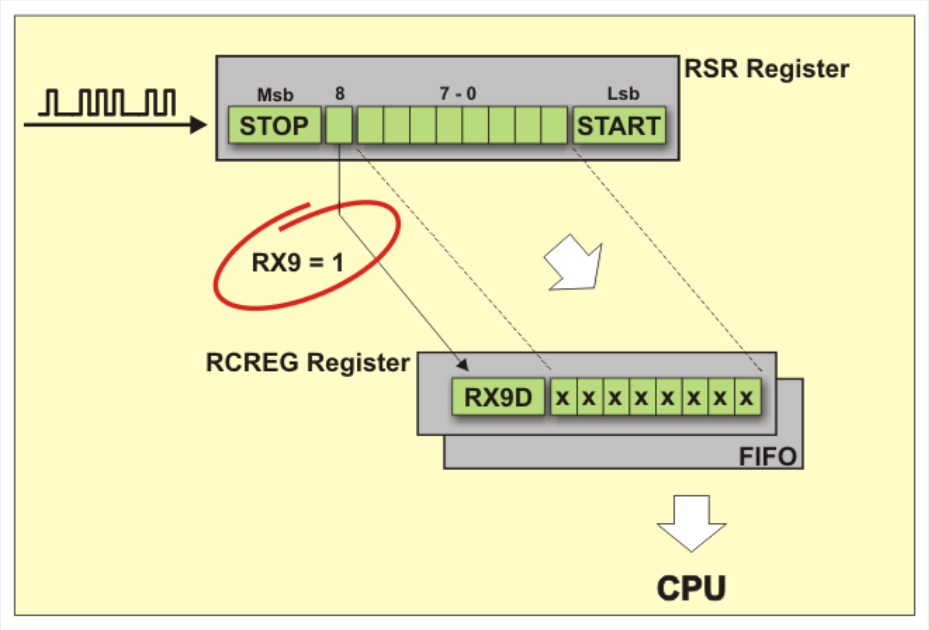
- A *Framing error* does not generate an interrupt by itself;
- If this bit is set, the last received data has an error;
- A framing error (bit set) does not prevent reception of new data;
- The FERR bit is cleared by reading received data, which means that check must be done prior to reading data; and
- The FERR bit cannot be cleared by software. If needed, it can be cleared by clearing the SPEN bit of the RCSTA register. It will simultaneously cause the whole EUSART system to be reset.

Another type of error is called *Overrun Error*. As previously mentioned, the FIFO memory can receive two characters only. An overrun error will be generated if the third character is received. Simply put, there is no space for another one byte and an error is unavoidable. When this happens the OERR bit of the RCSTA register is set. The consequences are the following:

- Data already stored in the FIFO registers (two bytes) can be normally read;
- No additional data will be received until the OERR bit is cleared; and
- This bit is not directly accessed. To clear it, it is necessary to clear the CREN bit of the RCSTAregister or reset the whole EUSART system by clearing the SPEN bit of the RCSTA register.

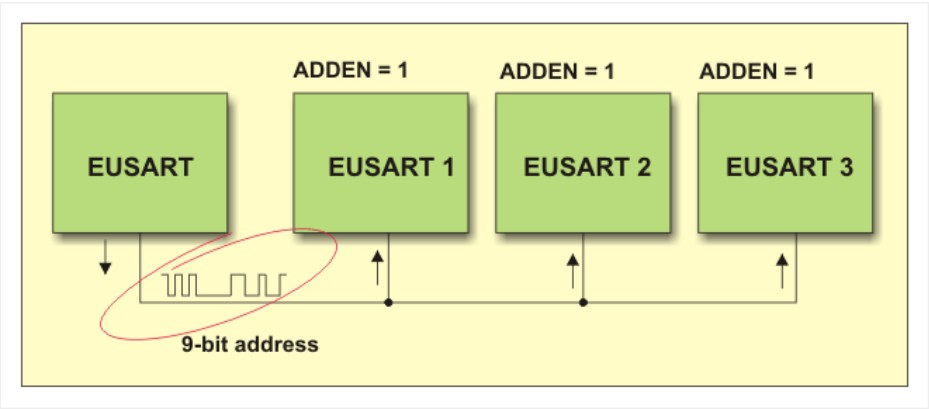
9-BIT DATA RECEIVE

Apart from receiving standard 8-bit data, the EUSART system supports 9-bit data reception. On the transmit side, the ninth bit is ‘attached’ to the original byte directly before the STOP bit. On the receive side, when the RX9 bit of the RCSTA register is set, the ninth data bit will be automatically written to the RX9D bit of the same register. After receiving this byte, it is necessary to take care of how to read its bits- the RX9D data bit must be read prior to reading 8 least significant bits of the RCREG register. Otherwise, the ninth data bit will be cleared.

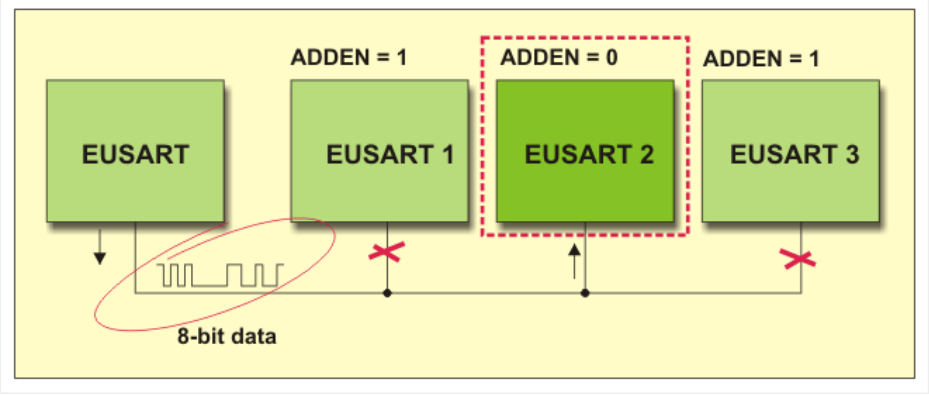


ADDRESS DETECTION

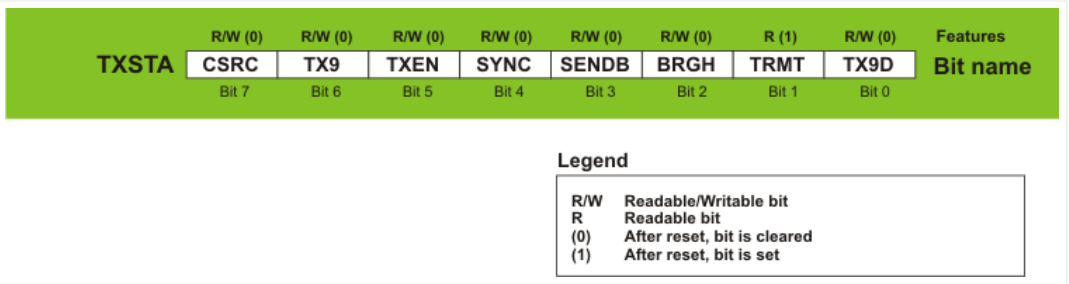
When the ADDEN bit of the RCSTA register is set, the EUSART module is able to receive only 9-bit data, whereas all 8-bit data will be ignored. Although it seems like a restriction, such modes enable serial communication between several microcontrollers. The principle of operation is simple. Master device sends a 9-bit data representing the address of one slave microcontroller. However, all of them must have the ADDEN bit set because it enables address detection. All slave microcontrollers, sharing the same transmission line, receive this data (address) and automatically check whether it matches their own address. Software, in which address match occurs, must disable address detection by clearing its ADDEN bit.



The master device keeps on sending 8-bit data. All data passing through the transmission line will be received by the addressed EUSART module only. When the last byte has been received, the slave device should set the ADDEN bit in order to enable new address detection.



TXSTA Register



CSRC – Clock Source Select bit – determines clock source. It is used only in synchronous mode.

- 1 – *Master* mode. Clock is generated internally from *Baud Rate* Generator.
- 0 – *Slave* mode. Clock is generated from external source.

TX9 – 9-bit Transmit Enable bit

- 1 – 9-bit data transmission via EUSART system.
- 0 – 8-bit data transmission via EUSART system.

TXEN – Transmit Enable bit

- 1 – Transmission enabled.
- 0 – Transmission disabled.

SYNC – EUSART Mode Select bit

- 1 – EUSART operates in synchronous mode.
- 0 – EUSART operates in asynchronous mode.

SENDB – Send Break Character bit is only used in asynchronous mode and when it is required to observe LIN bus standard.

- 1 – *Break* character transmission is enabled.
- 0 – *Break* character transmission is completed.

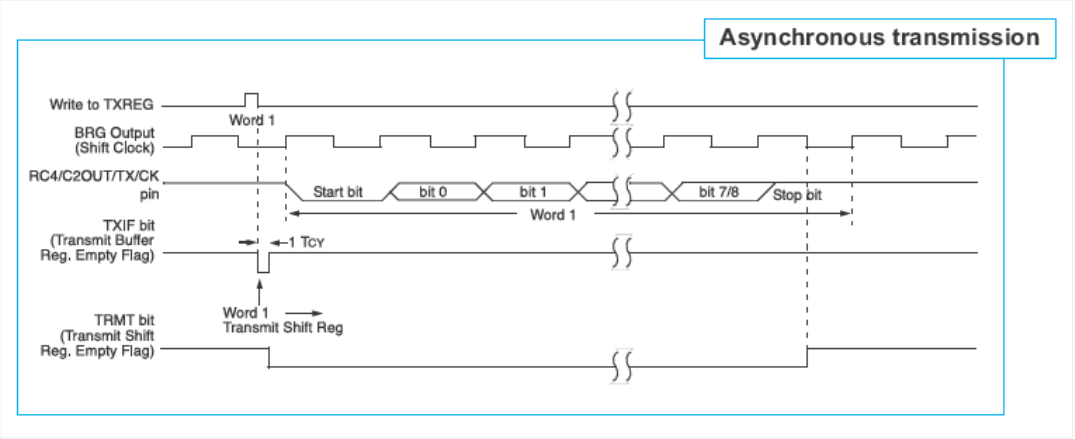
BRGH – High Baud Rate Select bit determines baud rate in asynchronous mode. It does not affect EUSART in synchronous mode.

- 1 – EUSART operates at high speed.
- 0 – EUSART operates at low speed.

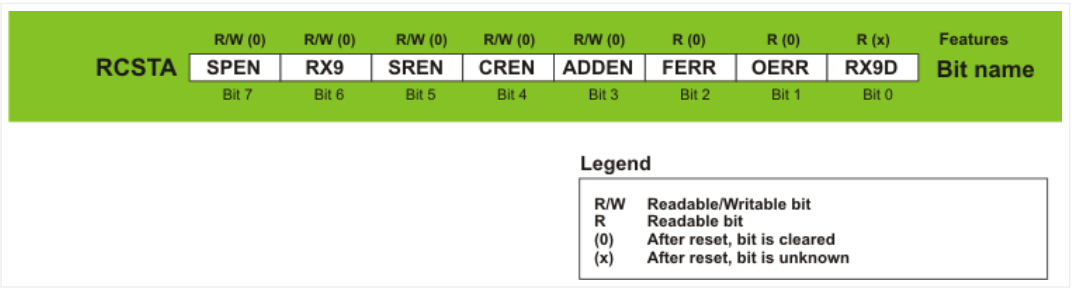
TRMT – Transmit Shift Register Status bit

- 1 – TSR register is empty.
- 0 – TSR register is full.

TX9D – Ninth bit of Transmit Data can be used as address or parity bit.



RCSTA Register



SPEN – Serial Port Enable bit

- 1 – Serial port enabled. RX/DT and TX/CK pins are automatically configured as input and output, respectively.
- 0 – Serial port disabled.

RX9 – 9-bit Receive Enable bit

- 1 – Reception of 9-bit data via EUSART system.
- 0 – Reception of 8-bit data via EUSART system.

SREN – Single ReceiveEnable bit is used only in synchronous mode when the microcontroller operates as *master*.

- 1 – Single receive enabled.
- 0 – Single receive disabled.

CREN – Continuous Receive Enable bit acts differently depending on EUSART mode.

Asynchronous mode:

- 1 – Receiver enabled.
- 0 – Receiver disabled.

Synchronous mode:

- 1 – Enables continuous receive until the CREN bit is cleared.
- 0 – Disables continuous receive.

ADDEN – Address Detect Enable bit is only used in address detect mode.

- 1 – Enables address detection on 9-bit data receive.
- 0 – Disables address detection. The ninth bit can be used as parity bit.

FERR – Framing Error bit

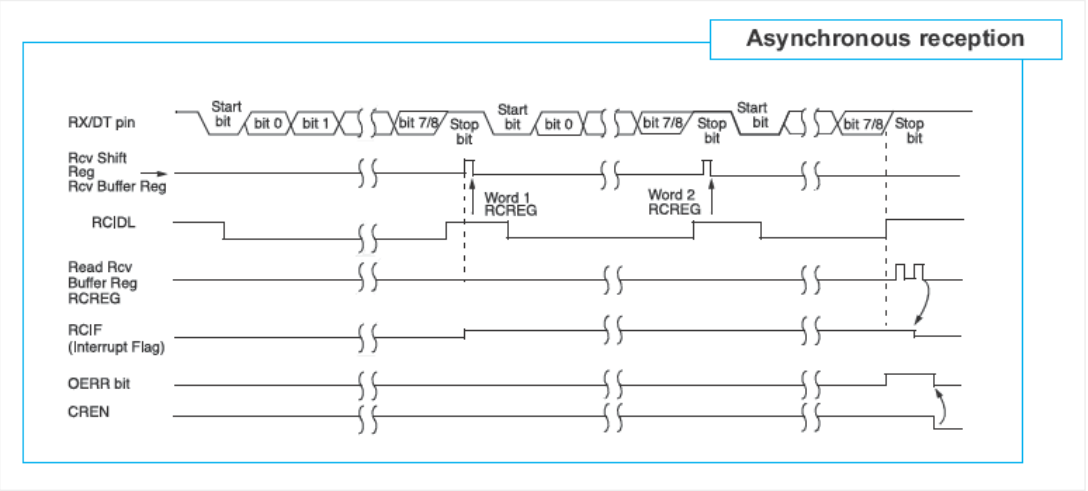
- 1 – On receive, *Framing Error* is detected.
- 0 – No framing error.

OERR – Overrun Error bit.

- 1 – On receive, *Overrun Error* is detected.
- 0 – No overrun error.

RX9D – Ninth bit of Received Data can be used as address or parity bit.

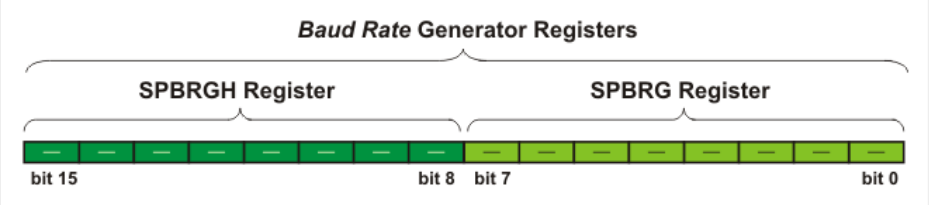
The next diagram shows three words appearing on the RX input. The receiving buffer is read after the third word, causing the OEER bit (overrun error bit) to be set.



EUSART BAUD RATE GENERATOR (BRG)

If you carefully look at asynchronous EUSART receiver or transmitter diagram, you will see that both of them use clock signal from the local timer BRG for synchronization. The same clock source is also used in synchronous mode.

The BRG timer consists of two 8-bit registers making one 16-bit register.



A number written to these two registers determines the baud rate. Besides, both the BRGH bit of the TXSTAregister and the BRGH16 bit of the BAUDCTL register affect clock frequency.

The formula used to determine *Baud Rate* is given in the table below.

BITS			BRG / EUSART MODE	BAUD RATE FORMULA
SYNC	BRG1G	BRGH		
0	0	0	8-bit / asynchronous	Fosc / [64 (n + 1)]
0	0	1	8-bit / asynchronous	Fosc / [16 (n + 1)]
0	1	0	16-bit / asynchronous	Fosc / [16 (n + 1)]
0	1	1	16-bit / asynchronous	Fosc / [4 (n + 1)]
1	0	X	8-bit / asynchronous	Fosc / [4 (n + 1)]
1	1	X	16-bit / asynchronous	Fosc / [4 (n + 1)]

Tables on the following pages contain values that should be written to the 16-bit register SPBRG and assigned to the SYNC, BRGH and BRGH16 bits in order to obtain some of the standard baud rates. Use the following formulas to determine the *Baud Rate*:

$$\text{Desired Baud Rate} = \frac{Fosc}{64(SPBRGH:SPBRG + 1)} \qquad SPBRGH:SPBRG = \frac{\frac{Fosc}{\text{Desired Baud Rate}}}{64}$$

$$\text{Error [\%]} = \frac{\text{Calc.Baud Rate - Desired Baud Rate}}{\text{Desired Baud Rate}}$$

Baud Rate	SYNC = 0, BRGH = 0, BRG16 = 0											
	Fosc = 20 MHz			Fosc = 18.432 MHz			Fosc = 11.0592 MHz			Fosc = 11.0592 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	-	-	-	-	-	-	-	-	-	-	-	-
1200	1221	1.73	255	1200	0.00	239	1200	0.00	143	1202	0.16	103
2400	2404	0.16	129	2400	0.00	119	2400	0.00	71	2404	0.16	51
9600	9470	-1.36	32	9600	0.00	29	9600	0.00	17	9615	0.16	12
10417	10417	0.00	29	10286	-1.26	27	10165	-2.42	16	10417	0.00	11
19.2k	19.53	1.73	15	19.2	0.00	14	19.2	0.00	8	-	-	-
57.6k	-	-	-	57.6k	0.00	7	57.6	0.00	2	-	-	-
115.2k	-	-	-	-	-	-	-	-	-	-	-	-

Baud Rate	SYNC = 0, BRGH = 0, BRG16 = 0											
	Fosc = 4 MHz			Fosc = 3.6864 MHz			Fosc = 2 MHz			Fosc = 1 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	300	0.16	207	300	0.00	191	300	0.16	103	300	0.16	51
1200	1202	0.16	51	1200	0.00	47	1202	0.16	25	1202	0.16	12
2400	2404	0.16	25	2400	0.00	23	2404	0.16	12	-	-	-
9600	-	-	-	9600	0.00	5	-	-	-	-	-	-
10417	10417	0.00	5	-	-	-	10417	0.00	2	-	-	-
19.2k	-	-	-	19.2	0.00	2	-	-	-	-	-	-
57.6k	-	-	-	57.6k	0.00	0	-	-	-	-	-	-
115.2k	-	-	-	-	-	-	-	-	-	-	-	-

Baud Rate	SYNC = 0, BRGH = 1, BRG16 = 0											
	Fosc = 20 MHz			Fosc = 18.432 MHz			Fosc = 11.0592 MHz			Fosc = 8 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	-	-	-	-	-	-	-	-	-	-	-	-
1200	-	-	-	-	-	-	-	-	-	-	-	-
2400	-	-	-	-	-	-	-	-	-	2404	0.16	207
9600	9615	0.16	129	9600	0.00	119	9600	0.00	71	9615	0.16	51
10417	10417	0.00	119	10378	-0.37	110	10473	0.53	65	10417	0.00	47
19.2k	19.23k	0.16	64	19.2	0.00	59	19.2k	0.00	35	19231	0.16	25
57.6k	56.82k	-1.36	21	57.6k	0.00	19	57.6k	0.00	11	55556	-3.55	8
115.2k	113.64k	-1.36	10	115.2k	0.00	9	115.2k	0.00	5	-	-	-

Baud Rate	SYNC = 0, BRGH = 1, BRG16 = 0											
	Fosc = 4 MHz			Fosc = 3.6864 MHz			Fosc = 2 MHz			Fosc = 1 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	-	-	-	-	-	-	-	-	-	300	0.16	207
1200	1202	0.16	207	1200	0.00	191	1202	0.16	103	1202	0.16	51
2400	2404	0.16	103	2400	0.00	95	2404	0.16	51	2404	0.16	25
9600	9615	0.16	25	9600	0.00	23	9615	0.16	12	-	-	-
10417	10417	0.00	23	10473	0.00	11	10417	0.00	11	10417	0.00	5
19.2k	19.23k	0.16	12	19.2	0.00	11	-	-	-	-	-	-
57.6k	-	-	-	57.6k	0.00	3	-	-	-	-	-	-
115.2k	-	-	-	115.2k	0.00	1	-	-	-	-	-	-

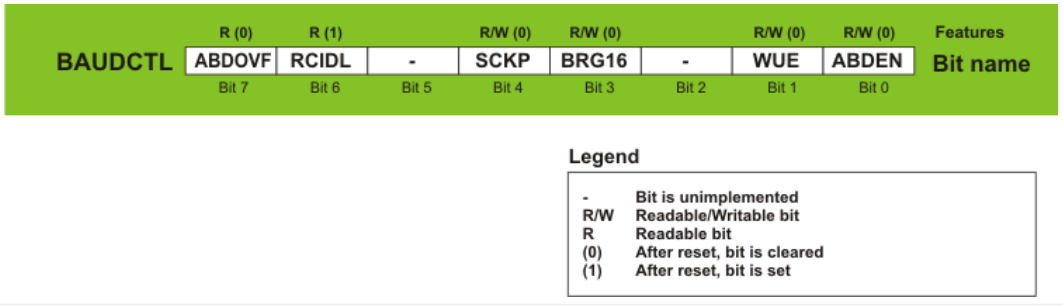
Baud Rate	SYNC = 0, BRGH = 0, BRG16 = 1											
	Fosc = 20 MHz			Fosc = 18.432 MHz			Fosc = 11.0592 MHz			Fosc = 8 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	300	-0.01	4166	300	0.00	3839	300	0.00	2303	299.9	-0.02	1666
1200	1200	-0.03	1041	1200	0.00	959	1200	0.00	575	1199	-0.08	416
2400	2399	-0.03	520	2400	0.00	479	2400	0.00	287	2404	0.16	207
9600	9615	0.16	129	9600	0.00	119	9600	0.00	71	9615	0.16	51
10417	10417	0.00	119	10378	-0.37	110	10473	0.53	65	10417	0.00	47
19.2k	19.23k	0.16	64	19.2k	0.00	59	19.2k	0.00	35	19.23k	0.16	25
57.6k	56.818	-1.36	21	57.6k	0.00	19	57.6k	0.00	11	55556	-3.55	8
115.2k	113.636	-1.36	10	115.2k	0.00	9	115.2k	0.00	5	-	-	-

Baud Rate	SYNC = 0, BRGH = 0, BRG16 = 1											
	Fosc = 4 MHz			Fosc = 3.6864 MHz			Fosc = 2 MHz			Fosc = 1 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	300.1	0.04	832	300	0.00	767	299.8	-0.108	416	300.5	0.16	207
1200	1202	0.16	207	1200	0.00	191	1202	0.16	103	1202	0.16	51
2400	2404	0.16	103	2400	0.00	95	2404	0.16	51	2404	0.16	25
9600	9615	0.16	25	9600	0.00	23	9615	0.16	12	-	-	-
10417	10417	0.00	23	10473	0.53	21	10417	0.00	11	10417	0.00	5
19.2k	19.23k	0.16	12	19.2k	0.00	11	-	-	-	-	-	-
57.6k	-	-	-	57.6	0.00	3	-	-	-	-	-	-
115.2k	-	-	-	115.2k	0.00	1	-	-	-	-	-	-

Baud Rate	SYNC = 0, BRGH = 1, BRG16 = 1 or SYNC = 1, BRGH16 = 1											
	Fosc = 20 MHz			Fosc = 18.432 MHz			Fosc = 11.0592 MHz			Fosc = 8 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	300	0.00	16665	300	0.00	15359	300	0.00	9215	300	0.00	6666
1200	1200	-0.01	4166	1200	0.00	3839	1200	0.00	2303	1200	-0.02	1666
2400	2400	0.02	2082	2400	0.00	1919	2400	0.00	1151	2401	0.04	832
9600	9597	-0.03	520	9600	0.00	479	9600	0.00	287	9615	0.16	207
10417	10417	0.00	479	10425	0.08	441	10433	0.16	264	10417	0	191
19.2k	19.23k	0.16	259	19.2k	0.00	239	19.2k	0.00	143	19.23k	0.16	103
57.6k	57.47k	-0.22	86	57.6k	0.00	79	57.6k	0.00	47	57.14k	-0.79	34
115.2k	116.3k	0.95	42	115.2k	0.00	39	115.2k	0.00	23	117.6k	2.12	16

Baud Rate	SYNC = 0, BRGH = 1, BRG16 = 1 or SYNC = 1, BRGH16 = 1											
	Fosc = 4 MHz			Fosc = 3.6864 MHz			Fosc = 2 MHz			Fosc = 1 MHz		
	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)	Actual Rate	Error %	SPBRG value (dec.)
300	300	0.01	3332	300	0.00	3071	299.9	-0.02	1666	300.1	0.04	832
1200	1200	0.04	832	1200	0.00	767	1199	-0.08	416	1202	0.16	207
2400	2398	0.08	416	2400	0.00	383	2404	0.16	207	2404	0.16	103
9600	9615	0.16	103	9600	0.00	96	9615	0.16	51	9615	0.16	25
10417	10417	0.00	95	10473	0.53	87	10417	0.00	47	10417	0.00	23
19.2k	19.23k	0.16	51	19.2k	0.00	47	19.23k	0.16	25	19.23k	0.16	12
57.6k	58.82k	2.12	16	57.6k	0.00	15	55.56k	-3.55	8	-	-	-
115.2k	111.1k	-3.55	8	115.2k	0.00	7	-	-	-	-	-	-

BAUDCTL Register



ABDOVF – Auto-Baud Detect Overflow bit is only used in asynchronous mode during baud rate detection.

- 1 – Auto-baud timer has overflowed.

- 0 – Auto-baud timer has not overflowed.

RCIDL – Receive Idle Flag bit is only used in asynchronous mode.

- 1 – Receiver is idle.
- 0 – START bit has been received and data receive is in progress.

SCKP – Synchronous Clock Polarity Select bit. The logic state of this bit varies depending on which EUSART mode is active.

Asynchronous mode:

- 1 – Transmit inverted data to the RC6/TX/CK pin.
- 0 – Transmit non-inverted data to the RC6/TX/CK pin.

Synchronous mode:

- 1 – Synchronization on the clock rising edge.
- 0 – Synchronization on the clock falling edge.

BRG16 16-bit Baud Rate Generator bit – determines whether the SPBRGH register will be used, i.e. whether the BRG timer will have 8 or 16 bits.

- 1 – 16-bit baud rate generator is used.
- 0 – 8-bit baud rate generator is used.

WUE Wake-up Enable bit

- 1 – Receiver waits for a falling edge on the RC7/RX/DT pin to wake up the microcontroller from sleep mode.
- 0 – Receiver operates normally.

ABDEN – Auto-Baud Detect Enable bit is used in asynchronous mode only.

- 1 – Auto-baud detect mode is enabled. Bit is automatically cleared on baud rate detection.
- 0 – Auto-baud detect mode is disabled.

Let's do it in mikroC…

```
/* In this example, internal EUSART module is initialized and set to send back the
message immediately after receiving it. Baude rate is set to 9600 bps. The program
uses UART library routines UART1_init(), UART1_Write_Text(), UART1_Data_Ready(),
UART1_Write() and UART1_Read().*/

char uart_rd;

void main() {
    ANSEL = ANSELH = 0;             <i>// Configure AN pins as digital</i>
    C1ON_bit = C2ON_bit = 0;        <i>// Disable comparators</i>
    UART1_Init(9600);               <i>// Initialize UART module at 9600 bps</i>
    Delay_ms(100);                  <i>// Wait for UART module to become stable</i>
    UART1_Write_Text("Start");

    while (1) {                     // Endless loop
        if (UART1_Data_Ready()) {   // If data is received,
            uart_rd = UART1_Read(); // read the received data,
            UART1_Write(uart_rd);   // and send data back via UART
        }`
    }
}
```

In Short

Data transmission via asynchronous EUSART communication:

1. The desired baud rate should be set by using bits BRGH (TXSTA register) and BRG16 (BAUDCTL register) and registers SPBRGH and SPBRG.
2. The SYNC bit (TXSTA register) should be cleared and the SPEN bit should be set (RCSTA register) in order to enable serial port.

3. The TX9 bit of the TXSTA register should be set on 9-bit data transmission.
4. Data transmission is enabled by setting the TXEN bit of the TXSTA register. The TXIF bit of the PIR1 register is automatically set.
5. The GIE and PEIE bits of the INTCON register should be set to enable the TXEN bit to cause an interrupt.
6. Value of the ninth bit should be written to the TX9D bit of the TXSTA register on 9-bit data transmission.
7. Transmission starts by writing 8-bit data to the TXREG register.

Data reception via asynchronous EUSART communication:

1. Baud Rate should be set by using bits BRGH (TXSTA register) and BRG16 (BAUDCTL register) and registers SPBRGH and SPBRG.
2. The SYNC bit (TXSTA register) should be cleared and the SPEN bit should be set (RCSTA register) in order to enable serial port.
3. Both the RCIE bit of the PIE1 register and bits GIE and PEIE of the INTCON register should be set when it is necessary to enable the data reception to cause an interrupt.
4. The RX9 bit of the RCSTA register should be set on 9-bit data receive.
5. Data reception is enabled by setting the CREN bit of the RCSTA register.
6. The RCSTA register should be read in order to check whether some errors have occurred during transmission. The ninth bit will be stored in this register on 9-bit data reception.
7. The received 8-bit data stored in the RCREG register should be read.

Setting Address Detection Mode:

1. Baud Rate should be set by using bits BRGH (TXSTA register) and BRG16 (BAUDCTL register) and registers SPBRGH and SPBRG.
2. The SYNC bit (TXSTA register) should be cleared and the SPEN bit should be set (RCSTA register) in order to enable serial port.
3. The RCIE bit of the PIE1 bit as well as bits GIE and PEIE of the INTCON register should be set when it is necessary to enable the data reception to cause an interrupt.
4. The RX9 bit of the RCSTA register should be set.
5. The ADDEN of the RCSTA register should be set, which enables data to be recognized as address.
6. Data reception should be enabled by setting the CREN bit of the RCSTA register.
7. As soon as the 9-bit data is received, the RCIF bit of the PIR1 register will be automatically set. If enabled, an interrupt occurs.
8. The RCSTA register should be read in order to check whether some errors have occurred during transmission. The ninth bit RX9D is always set.
9. The received 8-bit number stored in the RCREG register should be read. It should be checked whether the combination of these bits matches the predefined address. If the match occurs, it is necessary to clear the ADDEN bit of the RCSTA register, which enables 8-bit data to be received.

MASTER SYNCHRONOUS SERIAL PORT MODULE

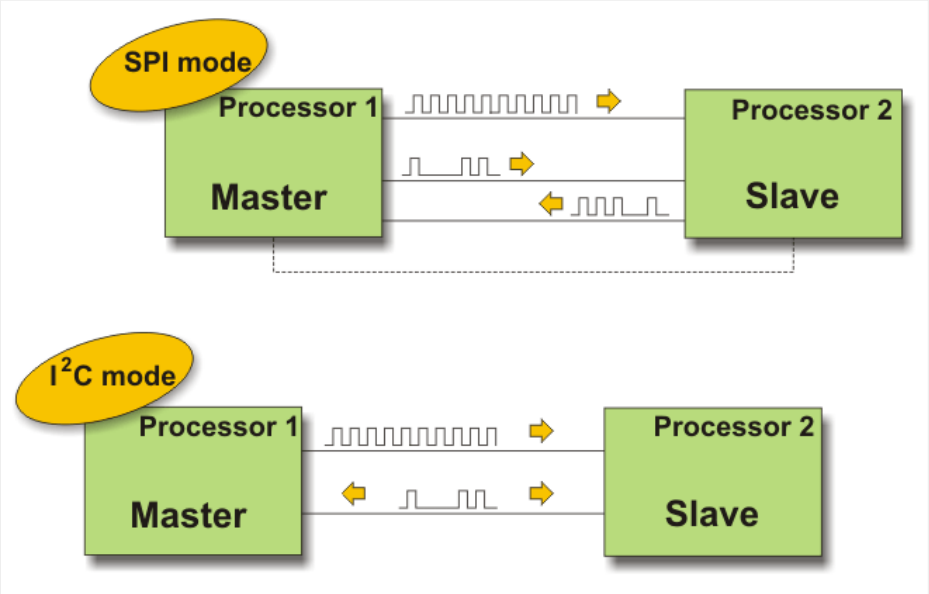
MSSP module (*Master Synchronous Serial Port*) is a very useful, but at the same time one of the most complex circuits within the microcontroller. It enables high speed communication between the microcontroller and other peripherals or other microcontrollers by using few input/output lines (maximum two or three). Therefore, it is commonly used to connect the microcontroller to LCD displays, A/D converters, serial EEPROMs, shift registers etc. The main feature of this type of communication is that it is synchronous and suitable for use in systems with a single master and one or more slaves. A master device contains a circuit for baud rate generation and supplies all

devices in the system with the clock. Slave devices may in this way eliminate the internal clock generation circuit. The MSSP module can operate in one out of two modes:

- SPI mode (*Serial Peripheral Interface*); and
- I²C mode (*Inter-Integrated Circuit*).

As seen in figure below, one MSSP module represents only a half of the hardware needed to establish serial communication, while the other half is stored in the device it exchanges data with. Even though the modules on both ends of the line are the same, their modes are essentially different depending on whether they operate as a *Master* or a *Slave*:

If the microcontroller to be programmed controls another device or circuit (peripherals), it should operate as a *master* device. It will generate clock when needed, i.e. only when data reception and transmission are required by the software. Obviously, connection establishment depends exclusively on the master device.



Otherwise, if the microcontroller to be programmed is integrated into a more complex device (for example, a PC) then it should operate as a*slave* device. As such, it always has to wait for data transmission request to be sent by the master device.

SPI MODE

The SPI mode allows 8 bits of data to be transmitted and received simultaneously using 3 input/output lines:

- **SDO** – *Serial Data Out* – transmit line;
- **SDI** – *Serial Data In* – receive line; and
- **SCK** – *Serial Clock* – synchronization line.

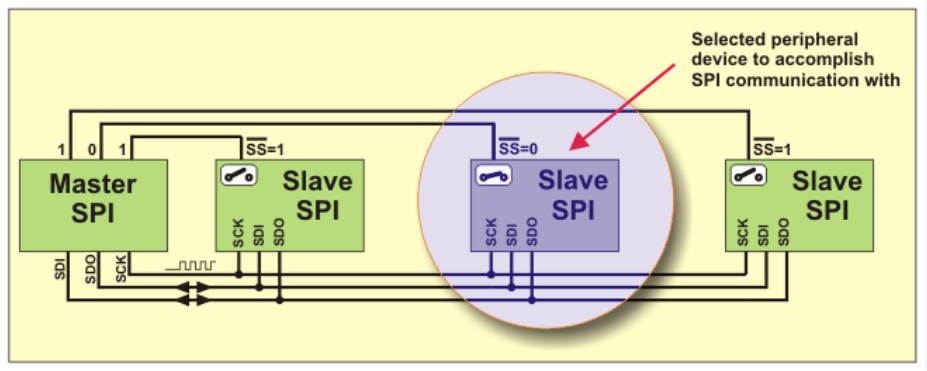
Apart from these three lines, there is the forth line (SS) as well which may be used if the microcontroller exchanges data with several peripheral devices. Refer to figure below.

SS – *Slave Select* – is additional pin used for specific device selection. It is active only when the microcontroller is in slave mode, i.e. when the external – master device requires data exchange.

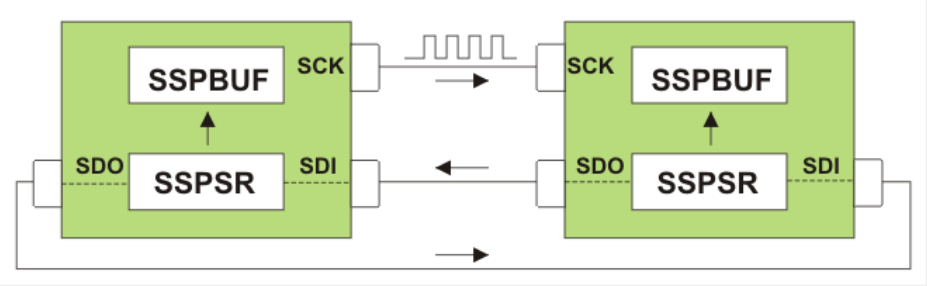
When operating in SPI mode, MSSP module uses in total of 4 registers:

- SSPSTAT – status register
- SSPCON – control register
- SSPBUF – buffer register
- SSPSR – shift register (not directly available)

The first three registers are writable/readable and can be changed at any moment, while the forth register, since not available, is used for converting data into ‘serial’ format.



As seen in figure below, the central part of the SPI module consists of two registers connected to pins for reception, transmission and synchronization.



The *Shift* register (SSPRS) is directly connected to the microcontroller pins and used for data transmission in serial format. The SSPRS register has its input and output so as to shift the data in and out of device. In other words, each bit appearing on the input (receive line) simultaneously shifts another bit toward the output (transmit line).

The SSPBUF register (*Buffer*) is part of memory used to temporarily hold the data prior to being sent or immediately after being received. After all 8 bits of data have been received, the byte is moved from the SSPRS to the SSPBUF register. This double buffering of the received data (SSPBUF) allows the next byte to start reception before reading the data that has just been received. Any write to the SSPBUF register during data transmission/ reception will be ignored. From the programmers’ point of view, this register is considered the most important as being most frequently accessed. Namely, if we neglect mode settings for a moment, data transfer via SPI actually comes to data write and read from this register, while another ‘acrobatics’ such as moving registers are automatically performed by hardware.

Let’s do it in mikroC…

```
/* In this example, PIC microcontroller (master) sends data byte to peripheral chip
(slave) via SPI. Program uses SPI library functions SPI1_init() and SPI1_Write. */

sbit Chip_Select at RC0_bit;              // Peripheral chip_select pin is connected to RC0
sbit Chip_Select_Direction at TRISC0_bit; // TRISC0 bit defines RC0 pin to be input or output
unsigned int value;                       // Data to be sent (value) is of unsigned int type

void main() {
    ANSEL = ANSELH = 0;                   // All I/O pins are digital
    TRISB0_bit = TRISB1_bit = 1;          // Configure RB0, RB1 pins as inputs
    Chip_Select = 0;                      // Select peripheral chip
    Chip_Select_Direction = 0;            // Configure the CS# pin as an output
    SPI1_Init();                          // Initialize SPI module
    SPI1_Write(value);                    // Send value to peripheral chip
    ...
```

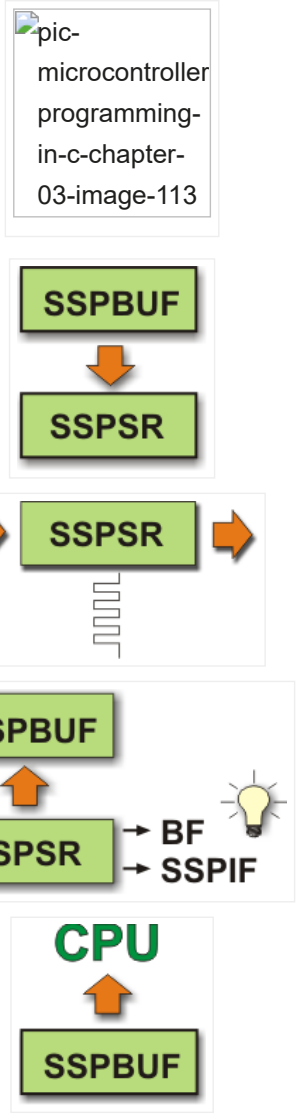
In short

Prior to the SPI initialization, it is necessary to specify several options:

- Master mode TRISC.3=0 (the SCK pin is the clock output);
- Slave mode TRISC.3=1 (the SCK pin is the clock input);

- Data input phase- middle or end of data output time (the SMP bit of the SSPSTAT register);
- Clock edge (the CKE bit of the SSPSTAT register);
- Baud Rate, bits SSPM3-SSPM0 of the SSPCON register (only in Master mode);
- Slave select mode, bits SSPM3-SSPM0 of the SSPCON register (Slave mode only).

The module starts to operate by setting the SSPEN bit:



Step 1.
Data to be transmitted should be written to the buffer register SSPBUF. If the SPI module operates in master mode, the microcontroller will automatically perform the following steps 2, 3 and 4. If the SPI module operates as Slave, the microcontroller will not perform these steps until the SCK pin detects clock signal.
Step 2.
The data is now moved to the SSPSR register and the SSPBUF register is not cleared.
Step 3.
This data is then shifted to the output pin (MSB bit first) while the register is simultaneously being filled with bits through the input pin. In Master mode, the microcontroller itself generates clock, while the Slave mode uses external clock (the SCK pin).
Step 4.
The SSPSR register is full once 8 bits of data have been received. It is indicated by setting the BF bit of the SSPSTAT register and the SSPIF bit of the PIR1 regis-ter. The received data (one byte) is automatically moved from the SSPSR register to the SSPBUF register. Since serial data transmission is performed automatically, the rest of the program is normally executed while the data transmission is in progress. In this case, the function of the SSPIF bit is to generate an interrupt when one byte transmission

is completed.

Step 5.

Finally, the data stored in the SSPBUF register is ready for use and should be moved to a desired register.

I²C MODE

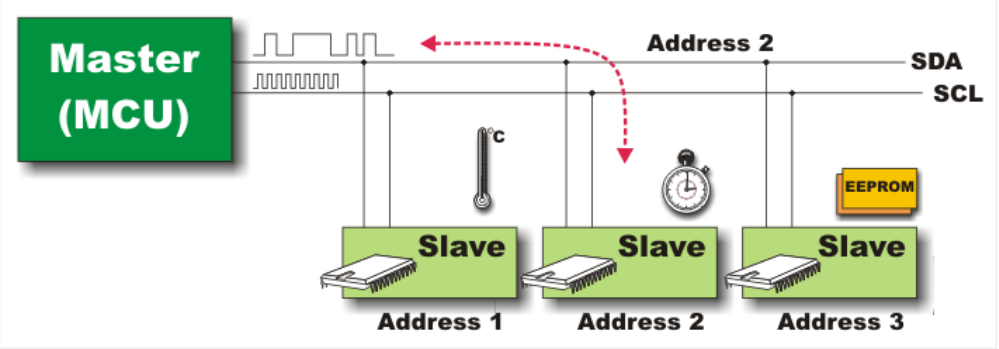
I²C mode (*Inter IC Bus*) is especially suitable when the microcontroller and an integrated circuit, which the microcontroller should exchange data with, are within the same device. It is usually another microcontrollers or specialized, cheap integrated circuits belonging to the new generation of so called 'smart peripheral components' (memories, temperature sensors, real-time clocks etc.)

Similar to serial communication in SPI mode, data transfer in I²C mode is synchronous and bidirectional. This time only two pins are used for data transmission. These are the SDA (Serial Data) and SCL (Serial Clock) pins. The user must configure these pins as inputs or outputs through the TRISC bits.

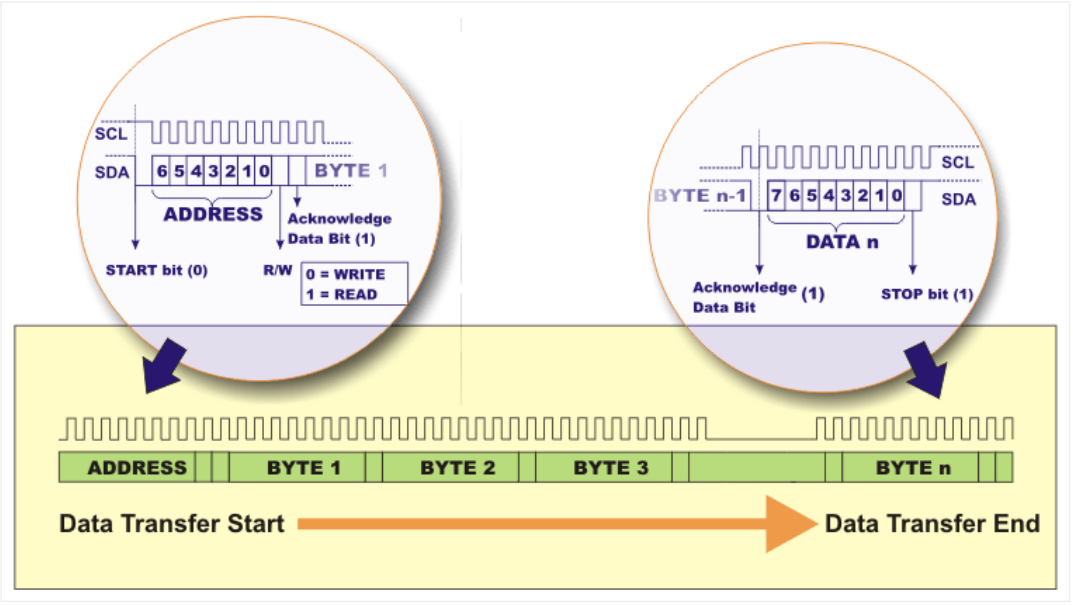
By observing particular rules (protocols), this mode enables up to 122 different components to be simultaneously connected in a simple way by using only two valuable I/O pins. Let's take a look at how it works:

Clock, necessary to synchronize the operation of both devices, is always generated by a master device (a microcontroller) and its frequency directly affects the baud rate. Even though there is a protocol allowing maximum 3,4 MHz clock frequency (so called highspeed I²C bus), this book covers only the most frequently used protocol the clock frequency of which is limited to 100 KHz. Minimum frequency is not limited.

When *master* and *slave* components are synchronized by the clock, every data exchange is always initiated by the *master*. Once the MSSP module has been enabled, it waits for a Start condition to occur. The master device first sends the START bit (logic zero) through the SDA pin, then a 7-bit address of the selected slave device, and finally, the bit which requires data write (0) or read (1) to the device. In other words, the eight bits are shifted to the SSPSR register following the start condition. All *slave* devices sharing the same transmission line will simultaneously receive the first byte, but only one of them has the address to match and receives the whole data.

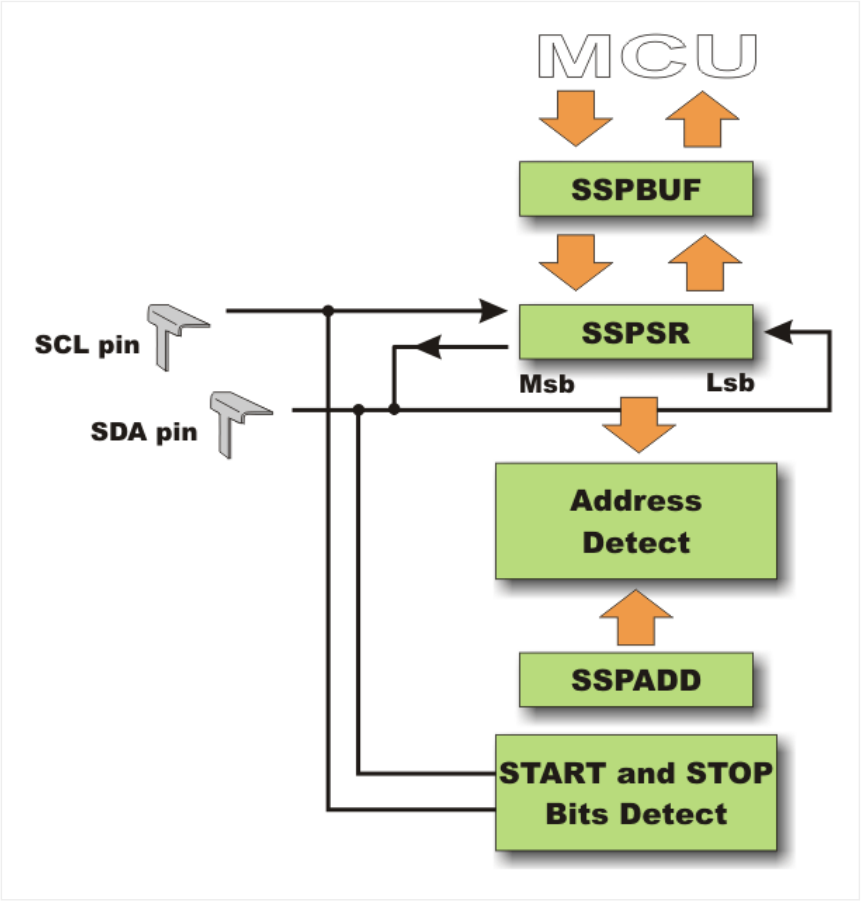


Once the first byte has been sent (only 8-bit data are transmitted), *master* goes into receive mode and waits for acknowledgment from the receive device that address match has occurred. If the *slave* device sends acknowledge data bit (1), data transfer will be continued until the*master* device (microcontroller) sends the Stop bit.



This is the simplest explanation of how two components communicate. Such a microcontroller is also capable of controlling more complicated situations when 1024 different components (10-bit address), shared by several different master devices, are connected. Such devices are rarely used in practice and there is no need to discuss them at greater length.

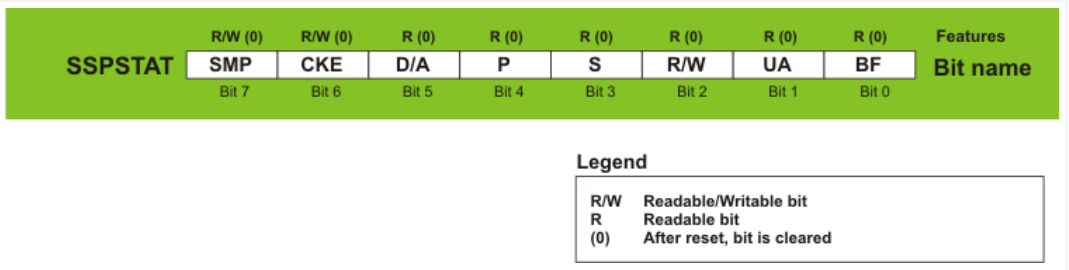
Figure below shows the block diagram of the MSSP module in I²C mode.



The MSSP module uses six registers for I²C operation. Some of them are shown in figure above:

- SSPCON
- SSPCON2
- SSPSTAT
- SSPBUF
- SSPSR
- SSPADD

SSPSTAT Register



SMP Sample bit

SPI master mode – This bit determines input data phase.

- 1 – Logic state is read at end of data output time.
- 0 – Logic state is read in the middle of data output time.

SPI slave mode – This bit must be cleared when SPI is used in Slave mode.

I²C mode (*master* or *slave*)

- 1 – Slew rate control disabled for standard speed mode (100kHz).
- 0 – Slew rate control enabled for high speed mode (400kHz).

CKE – Clock Edge Select bit selects synchronization mode.

CKP = 0:

- 1 – Data is transmitted on rising edge of clock pulse (0 – 1).
- 0 – Data is transmitted on falling edge of clock pulse (1 – 0).

CKP = 1:

- 1 – Data is transmitted on falling edge of clock pulse (1 – 0).
- 0 – Data is transmitted on rising edge of clock pulse (0 – 1).

D/A – Data/Address bit is used in I²C mode only.

- 1 – Indicates that the last byte received or transmitted was data.
- 0 – Indicates that the last byte received or transmitted was address.

P – Stop bit is used in I²C mode only.

- 1 – STOP bit was detected last.
- 0 – STOP bit was not detected last.

S – Start bit is used in I²C mode only.

- 1 – START bit was detected last.
- 0 – START bit was not detected last.

R/W – Read Write bit is used in I²C mode only. This bit holds the R/W bit information following the last address match. This bit is only valid from the address match to the next Start bit, Stop bit or not ACK bit.

In I²C slave mode

- 1 – Data read.
- 0 – Data write.

In I²C master mode

- 1 – Transmit is in progress.

- 0 – Transmit is not in progress.

UA – Update Address bit is used in 10-bit I2C mode only.

- 1 – The SSPADD register must be updated.
- 0 – Address in the SSPADD register is correct and doesn't need to be updated.

BF Buffer Full Status bit

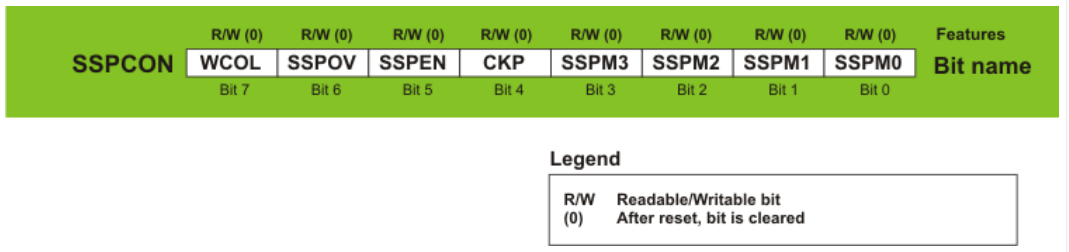
During data receive (in SPI and I²C modes)

- 1 – Receive complete. The SSPBUF register is full.
- 0 – Receive not complete. The SSPBUF register is empty.

During data transmit (in I²C mode only)

- 1 – Data transmit in progress (doesn't include the ACK and STOP bits).
- 0 – Data transmit complete (doesn't include the ACK and STOP bits).

SSPCON Register



WCOL Write Collision Detect bit

- 1 – Collision detected. Write to the SSPBUF register was attempted while the I²C conditions were not valid for transmission to start.
- 0 – No collision.

SSPOV Receive Overflow Indicator bit

- 1 – A new byte is received before reading the previously received data. Since there is no space for new data receive, one of these two bytes must be cleared. In this case, data stored in the SSPSR register is irretrievably lost.
- 0 – Serial data is correctly received.

SSPEN – Synchronous Serial Port Enable bit determines the microcontroller pins function and initializes MSSP module:

In SPI mode

- 1 – Enables MSSP module and configures pins SCK, SDO, SDI and SS as the source of the serial port pins.
- 0 – Disables MSSP module and configures these pins as I/O port pins.

In I²C mode

- 1 – Enables MSSP module and configures pins SDA and SCL as the source of the serial port pins.
- 0 – Disables MSSP module and configures these pins as I/O port pins.

CKP – Clock Polarity Select bit is not used in I²C master mode.

In SPI mode

- 1 – Idle state for clock is a high level.
- 0 – Idle state for clock is a low level.

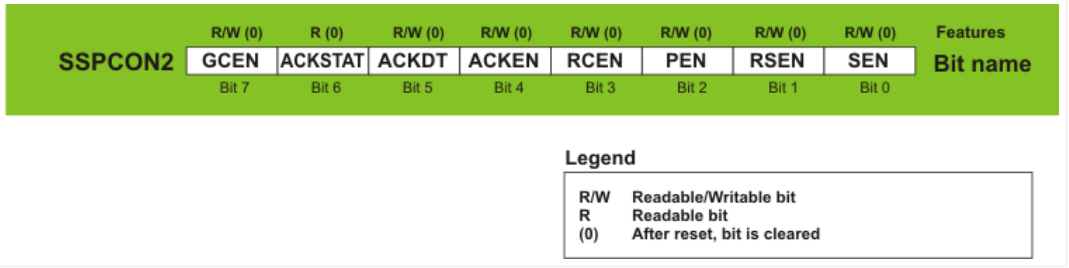
In I²C slave mode

- 1 – Enables clock.
- 0 – Holds clock low. Used to provide more time for data stabilization.

SSPM3-SSPM0 – Synchronous Serial Port Mode Select bits. SSP mode is determined by combining these bits:

SSPM3	SSPM2	SSPM1	SSPM0	MODE
0	0	0	0	SPI master mode, clock = Fosc/4
0	0	0	1	SPI master mode, clock = Fosc/16
0	0	1	0	SPI master mode, clock = Fosc/64
0	0	1	1	SPI master mode, clock = (output TMR)/2
0	1	0	0	SPI slave mode, SS pin control enabled
0	1	0	1	SPI slave mode, SS pin control disabled, SS can be used as I/O pin
0	1	1	0	I²C slave mode, 7-bit address used
0	1	1	1	I²C slave mode, 10-bit address used
1	0	0	0	I²C master mode, clock = Fosc / [4(SSPAD+1)]
1	0	0	1	Mask used in I²C slave mode
1	0	1	0	Not used
1	0	1	1	I²C controlled master mode
1	1	0	0	Not used
1	1	0	1	Not used
1	1	1	0	I²C slave mode, 7-bit address used,START and STOP bits enable interrupt
1	1	1	1	I²C slave mode, 10-bit address used,START and STOP bits enable interrupt

SSPCON2 Register



GCEN – General Call Enable bit

In I²C slave mode only

- 1 – Enables interrupt when a general call address (0000h) is received in the SSPSR.
- 0 – General call address disabled.

ACKSTAT – Acknowledge Status bit

In I²C Master Transmit mode only

- 1 – Acknowledge was not received from slave.
- 0 – Acknowledge was received from slave.

ACKDT – Acknowledge data bit

In I²C Master Receive mode only

- 1 – Not Acknowledge.

- 0 – Acknowledge.

ACKEN – Acknowledge Sequence Enable bit

In I²C Master Receive mode

- 1 – Initiate acknowledge condition on the SDA and SCL pins and transmit the ACKDT data bit. It is automatically cleared by hardware.
- 0 – Acknowledge condition is not initiated.

RCEN – Receive Enable bit

In I²C Master mode only

- 1 – Enables data receive in I²C mode.
- 0 – Receive disabled.

PEN – STOP condition Enable bit

In I²C Master mode only

- 1 – Initiates STOP condition on the SDA and SCL pins. Afterwards, this bit is automatically cleared by hardware.
- 0 – STOP condition is not initiated.

RSEN – Repeated START Condition Enabled bit

In I²C master mode only

- 1 – Initiates START condition on the SDA and SCL pins. Afterwards, this bit is automatically cleared by hardware.
- 0 – Repeated START condition is not initiated.

SEN – START Condition Enabled/Stretch Enabled bit

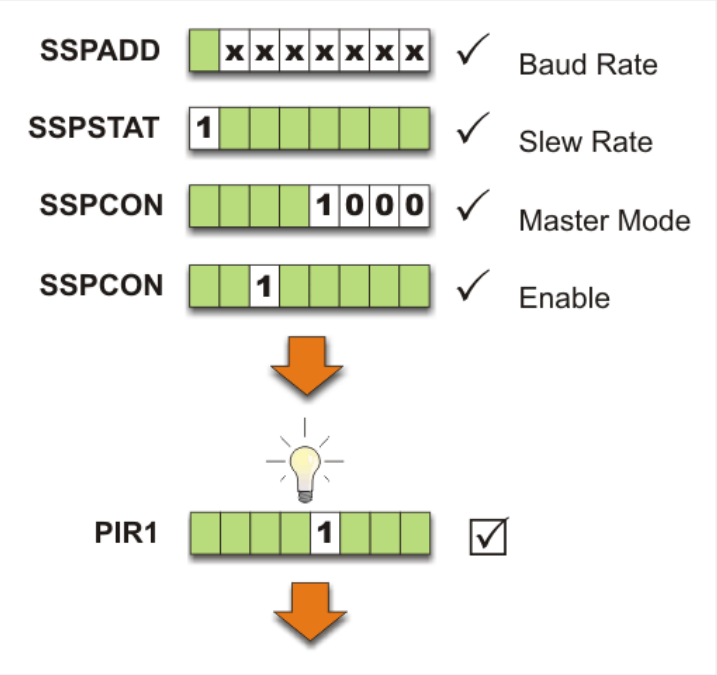
In I²C Master mode only

- 1 – Initiates START condition on the SDA and SCL pins. Afterwards, this bit is automatically cleared by hardware.
- 0 – START condition is not initiated.

I²C in Master Mode

The most common case is that the microcontroller operates as a *master* and a peripheral component as a *slave*. This is why this book covers just this mode. It is also considered that the address consists of 7 bits and device contains only one microcontroller (*single-master*device).

In order to enable MSSP module in this mode, it is necessary to do the following:

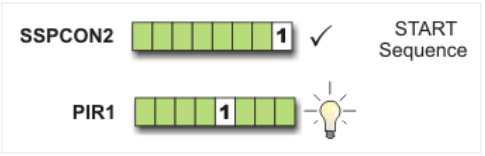


Set baud rate (SSPADD register), turn off *slew rate* control (by setting the SMP bit of the SSPSTAT register) and select *master* mode (SSPCON register). After all these preparations have been finished and the module has been enabled (SSPCON register : SSPEN bit), it is necessary to wait for internal electronics to signal that everything is ready for data transmission, i.e. the SSPIF bit of the PIR1 register is set.

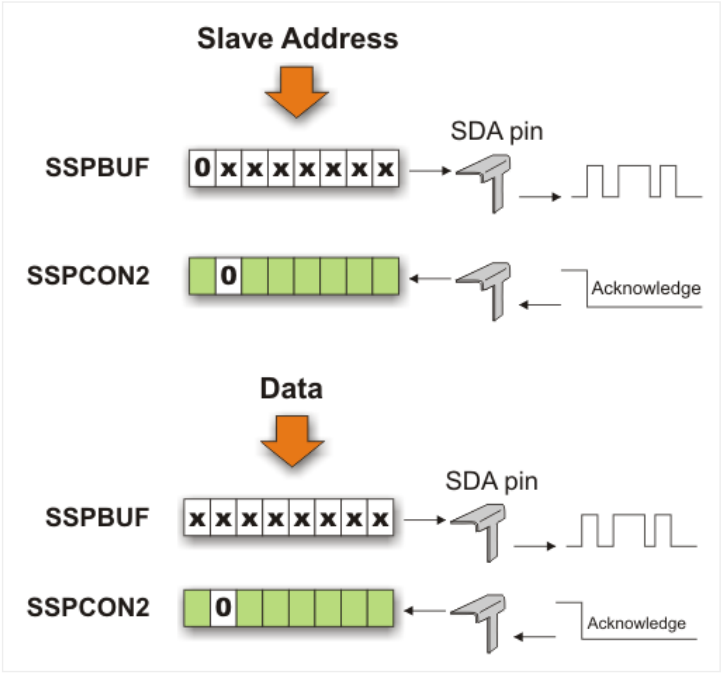
This bit should be cleared by software and after that the microcontroller is ready to exchange data with peripherals.

Data Transmission in I2C Master Mode

Data transmission on the SDA pin starts with a logic zero (0) which appears upon setting the SEN bit of the SSPCON2 register. Even enabled, the microcontroller has to wait a certain time before it starts communication. It is the so called ‘Start condition’ during which internal preparations and checks are performed. If all conditions are met, the SSPIF bit of the PIR1 is set and data transmission starts as soon as the SSPBUF register is loaded.



Maximum 112 integrated circuits (*slave* devices) may simultaneously share the same transmission line. The first data byte sent by the*master* device contains the address to match only one slave device. All addresses are listed in respective data sheets. The eighth bit of the first data byte specifies direction of data transmission, i.e. whether the microcontroller is to send or receive data. In this case, the eighth bit is cleared to logic zero (0), which means that it is data transmission.



When address match occurs, the microcontroller has to wait for the acknowledge data bit. The slave device acknowledges address match by clearing the ASKSTAT bit of the SSPCON2 register. If the match properly occurred, all data bytes are transmitted in the same way.

Data transmission ends by setting the SEN bit of the SSPCON2 register. The STOP condition occurs, which enables the SDA pin to receive pulse condition:

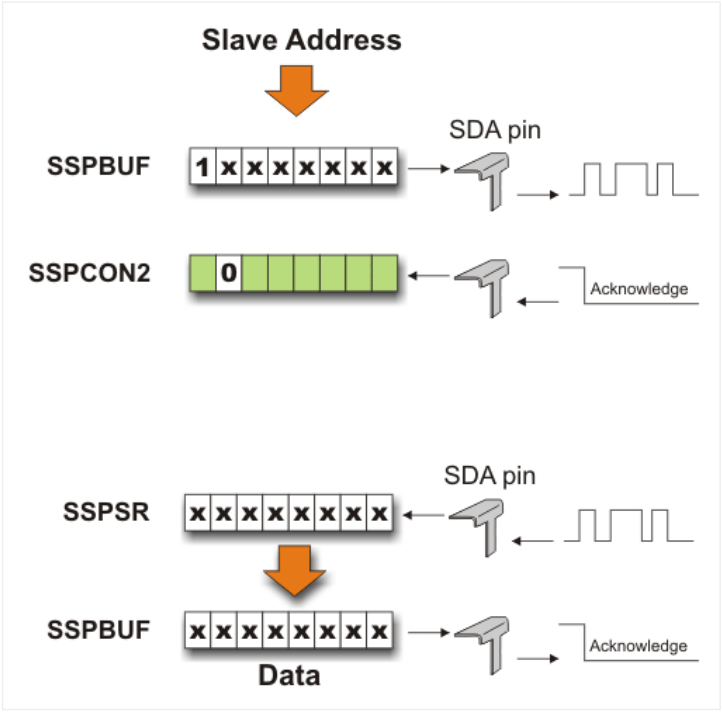
Start – Address – Acknowledge – Data – Acknowledge….Data – Acknowledge – Stop!

Data Reception in I²C Master Mode

Preparations for data reception are similar to those for data transmission, with exception that the last bit of the first sent byte (containing address) is set to logic one (1). It specifies that *master* expects to receive data from the addressed slave device. In relation to the microcontroller, the following occurs:

After internal preparations are finished and the START bit is set, *slave* device starts sending one byte at a time. These bytes are stored in the serial register SSPSR. Each data is, after receiving the last eighth bit, loaded to the SSPBUF register from where it can be read. Reading this register causes the acknowledge bit to be automatically sent, which means that the master device is ready to receive new data.

Likewise, data reception ends by setting the STOP bit:

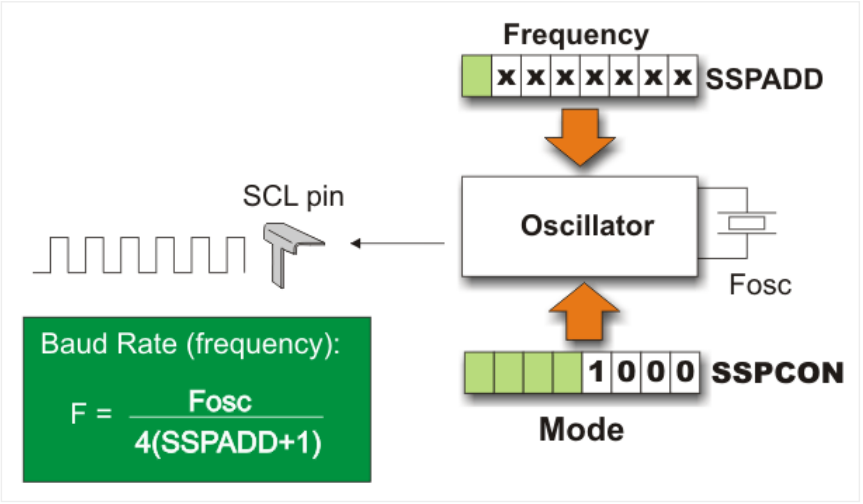


Start – Address – Acknowledge – Data – Acknowledge….Data – Acknowledge – Stop!

In this pulse sequence, the acknowledge bit is sent to *slave* device.

In order to synchronize data transmission, all events taking place on the SDA pin must be synchronized with the clock generated in the master device. This clock is generated by a simple oscillator the frequency of which depends on the microcontroller's main oscillator frequency, the value written to the SSPADD register and the current SPI mode as well.

The clock frequency of the mode described in this book depends on selected quartz crystal and the SPADD register. Figure below shows the formula used to calculate it.



Let's do it in mikroC…

```
// In this example, PIC MCU is connected to 24C02 EEPROM via SCL and SDA pins. The program
// sends one byte of data to the EEPROM address 2. Then, it reads that data via I2C from
// EEPROM and sends it to PORTB in order to check if the data was successfully written. */

void main(){
    ANSEL = ANSELH = PORTB = TRISB = 0; // All pins are digital. PORTB pins are outputs.
    I2C1_Init(100000);                  // Initialize I2C with desired clock
    I2C1_Start();                       // I2C start signal
    I2C1_Wr(0xA2);                      // Send byte via I2C (device address + W)
    I2C1_Wr(2);                         // Send byte (address of EEPROM location)
    I2C1_Wr(0xF0);                      // Send data to be written
    I2C1_Stop();                        // I2C stop signal

    Delay_100ms();

    I2C1_Start();                       // I2C start signal
    I2C1_Wr(0xA2);                      // Send byte via I2C (device address + W)
    I2C1_Wr(2);                         // Send byte (data address)
```

```
19      I2C1_Repeated_Start();              <i>// Issue I2C signal repeated start</i>
20      I2C1_Wr(0xA3);                      <i>// Send byte (device address + R)</i>
21      PORTB = I2C1_Rd(0u);                <i>// Read the data (NO acknowledge)</i>
22      I2C1_Stop();                        <i>// I2C stop signal</i>
23  }
```

USEFUL NOTES …

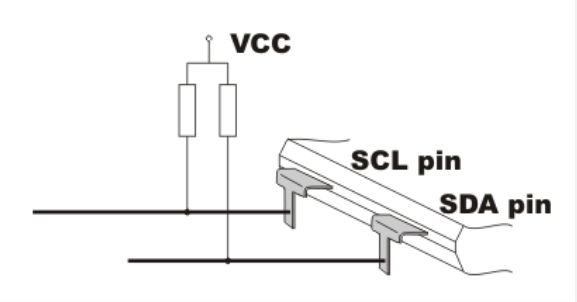
When the microcontroller communicates with peripheral components, it may happen that data transmission fails for some reason. In that case, it is recommended to check the state of some of the bits which can clarify the problem. In practice, the status of these bits is checked by executing a short subroutine after each byte transmission and reception (just in case).

WCOL (SPCON,7) – If you try to write a new data to the SSPBUF register while another data transmission/reception is in progress, the WCOL bit will be set and the contents of the SSPBUF register remains unchanged. Write does not occur. After this, the WCOL bit must be cleared in software.

BF (SSPSTAT,0) – In transmission mode, this bit is set when the CPU writes to the SSPBUF register and remains set until the byte in serial format is shifted from the SSPSR register. In reception mode, this bit is set when data or address is loaded to the SSPBUF register. It is cleared after reading the SSPBUF register.

SSPOV (SSPCON,6) – In reception mode, this bit is set when a new byte is received by the SSPSR register via serial communication, whereas the previously received data has not been read from the SSPBUF register yet.

SDA and SCL Pins – When SPP module is enabled, these pins turn into *Open Drain* outputs. It means that they must be connected to the resistors which are by the other end connected to positive power supply.



In short

In order to establish serial communication in I2C mode, the following should be done:

Setting Module and Sending Address:

- Value to determine baud rate should be written to the SSPADD register.
- SlewRate control should be turned off by setting the SMP bit of the SSPSTAT register.
- In order to select Master mode, binary value 1000 should be written to the SSPM3-SSPM0 bits of the SSPCON1 register.
- The SEN bit of the SSPCON2 register (START sequence) should be set.
- The SSPIF bit is automatically set at the end of START sequence when the module is ready to operate. It should be cleared.
- Slave address should be written to the SSPBUF register.
- When the byte is sent, the SSPIF bit (interrupt) is automatically set after receiving the acknowledge bit from the Slave device.

Data Transmit:

- Data to be send should be written to the SSPBUF register.
- When the byte is sent, the SSPIF bit (interrupt) is automatically set after receiving the acknowledge bit from Slave device.

- In order to inform the Slave device that data transmission is complete, STOP condition should be initiated by setting the PEN bit of the SSPCON register.

Data Receive:

- In order to enable reception, the RSEN bit of the SSPCON2 register should be set.
- The SSPIF bit signals data reception. When data is read from the SSPBUF register, the ACKEN bit of the SSPCON2 register should be set in order to enable acknowledge bit to be sent.
- In order to inform the Slave device that data transmission is complete, the STOP condition should be initiated by setting the PEN bit of the SSPCON register.

In addition to a large number of digital I/O lines used for communication with peripherals, the PIC16F887 contains 14 analog inputs. They enable the microcontroller to recognize not only whether a pin is driven to logic zero or one (0 or +5V), but to precisely measure its voltage and convert it into numerical value, i.e. digital format.